ONEDRAFT

CAD-AI — Aria Powered

Database Schema & OpenAPI Contract

Version 0.1

Status: Implementation Specification
Database: PostgreSQL
API: REST / JSON
API Version: `/api/v1`
Identifiers: UUID
Primary Model: Versioned OneDraft Project Model
AI Layer: Aria Powered

---

1. DATABASE DESIGN PRINCIPLE

PostgreSQL is the authoritative transactional persistence layer for OneDraft.

The database shall preserve four distinct concepts:

Current state

The currently active project model.

Historical state

Previous model versions and revisions.

Derived state

Drawings, schedules, validations and generated documents.

Event state

The immutable record of what occurred.

These states must not be conflated.

The database therefore becomes the persistent computational memory of the OneDraft platform.

---

2. DATABASE SCHEMAS

The PostgreSQL database shall initially use the following logical schemas:

```
identity
projects
model
geometry
documentation
compliance
validation
revisions
audit
billing
system
```

The primary relationships are:

```
identity
    ↓
projects
    ↓
model
    ↓
geometry
    ↓
documentation
    ↓
validation
    ↓
revisions
    ↓
audit
```

---

3. UNIVERSAL DATABASE RULES

All primary keys use:

`UUID`

All timestamps use:

`TIMESTAMPTZ`

All timestamps are stored in UTC.

All mutable records contain:

```
created_at
updated_at
```

Where applicable:

```
created_by
```

```
updated_by
```

Soft deletion should be used for user-facing project objects rather than immediate physical deletion.

Example:

```
status = DELETED
deleted_at = timestamp
```

Historical records are never physically deleted as part of normal application operation.

---

4. ORGANIZATION TABLE

```
identity.organizations
```

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(255) NOT NULL
organization_type VARCHAR(64)
status VARCHAR(32) NOT NULL
billing_account_id UUID
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_organizations_status
idx_organizations_billing
```

---

5. USER TABLE

```
identity.users
```

Fields:

```
id UUID PRIMARY KEY
email VARCHAR(320) NOT NULL
display_name VARCHAR(255)
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(email)
```

---

6. ORGANIZATION MEMBERSHIP

`identity.organization_members`

Fields:

```
organization_id UUID NOT NULL
user_id UUID NOT NULL
role VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Primary key:

```
(organization_id, user_id)
```

Foreign keys:

```
organization_id → organizations.id
user_id → users.id
```

---

7. PROJECT TABLE

`projects.projects`

Fields:

```
id UUID PRIMARY KEY
organization_id UUID NOT NULL
name VARCHAR(255) NOT NULL
project_type VARCHAR(64)
status VARCHAR(32) NOT NULL
description TEXT
address JSONB
jurisdiction JSONB
unit_system VARCHAR(32) NOT NULL
current_revision_id UUID
current_model_version_id UUID
current_code_manifest_id UUID
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_projects_organization
idx_projects_status
idx_projects_updated
```

The project is the primary tenancy boundary beneath the organization.

---

8. PROJECT REQUIREMENTS

`projects.requirements`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
source VARCHAR(64) NOT NULL
description TEXT NOT NULL
requirement_type VARCHAR(64) NOT NULL
priority INTEGER DEFAULT 0
mandatory BOOLEAN DEFAULT FALSE
status VARCHAR(32) NOT NULL
verification_status VARCHAR(32) NOT NULL
affected_objects JSONB
created_revision UUID
modified_revision UUID
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_requirements_project
idx_requirements_type
idx_requirements_status
```

---

9. SITE TABLE

`model.sites`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL UNIQUE
boundary_geometry JSONB
north_angle NUMERIC
elevation_reference VARCHAR(128)
topography JSONB
existing_conditions JSONB
site_constraints JSONB
setbacks JSONB
easements JSONB
rights_of_way JSONB
utilities JSONB
parking JSONB
access JSONB
survey_reference VARCHAR(1024)
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

The MVP stores normalized site metadata while permitting complex geometry to be stored through the geometry subsystem.

---

10. BUILDINGS

`model.buildings`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
name VARCHAR(255) NOT NULL
building_type VARCHAR(64)
occupancy VARCHAR(128)
construction_type VARCHAR(128)
height NUMERIC
area NUMERIC
number_of_levels INTEGER
geometry_reference_id UUID
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_buildings_project
idx_buildings_status
```

---

11. LEVELS

`model.levels`

Fields:

```
id UUID PRIMARY KEY
building_id UUID NOT NULL
name VARCHAR(128) NOT NULL
level_code VARCHAR(64) NOT NULL
elevation NUMERIC NOT NULL
floor_to_floor_height NUMERIC
finished_floor_elevation NUMERIC
ceiling_elevation NUMERIC
sequence INTEGER NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(building_id, level_code)
```

Index:

```
idx_levels_building_sequence
```

---

12. GRIDS

`model.grids`

Fields:

```
id UUID PRIMARY KEY
building_id UUID NOT NULL
grid_type VARCHAR(32) NOT NULL
label VARCHAR(64) NOT NULL
geometry_reference_id UUID
sequence INTEGER
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

13. SPACES

`model.spaces`

Fields:

```
id UUID PRIMARY KEY
level_id UUID NOT NULL
space_number VARCHAR(64)
name VARCHAR(255)
space_type VARCHAR(64)
occupancy VARCHAR(128)
area NUMERIC
volume NUMERIC
boundary_geometry_reference_id UUID
ceiling_height NUMERIC
accessibility_class VARCHAR(64)
fire_classification VARCHAR(64)
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_spaces_level
idx_spaces_type
```

---

14. ELEMENTS

`model.elements`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
level_id UUID
parent_id UUID
element_type VARCHAR(64) NOT NULL
name VARCHAR(255)
geometry_reference_id UUID
assembly_id UUID
host_element_id UUID
parameters JSONB NOT NULL DEFAULT '{}'
status VARCHAR(32) NOT NULL
created_revision_id UUID
modified_revision_id UUID
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_elements_project
idx_elements_level
idx_elements_parent
idx_elements_type
idx_elements_host
idx_elements_status
```

This becomes one of the most heavily accessed tables in the system.

---

15. ASSEMBLIES

`model.assemblies`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
name VARCHAR(255) NOT NULL
assembly_type VARCHAR(64)
layers JSONB NOT NULL
total_thickness NUMERIC
performance_properties JSONB
fire_rating VARCHAR(64)
thermal_properties JSONB
acoustic_properties JSONB
manufacturer_data JSONB
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

16. CONSTRAINTS

`model.constraints`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
constraint_type VARCHAR(64) NOT NULL
source VARCHAR(64) NOT NULL
priority INTEGER DEFAULT 0
value JSONB
unit VARCHAR(32)
tolerance JSONB
affected_objects JSONB NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

The value field uses JSONB because constraints may contain scalar, vector, interval, relational or compound data.

---

17. RELATIONSHIPS

`model.relationships`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
source_object_id UUID NOT NULL
target_object_id UUID NOT NULL
relationship_type VARCHAR(64) NOT NULL
metadata JSONB
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_relationships_source
idx_relationships_target
idx_relationships_project_type
```

This table forms the initial dependency graph.

---

18. GEOMETRY REFERENCES

`geometry.geometry_references`

Fields:

```
id UUID PRIMARY KEY
object_id UUID NOT NULL
geometry_type VARCHAR(32) NOT NULL
coordinate_system VARCHAR(128)
geometry_version INTEGER NOT NULL
storage_reference TEXT NOT NULL
bounding_box JSONB
geometry_hash VARCHAR(128) NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(object_id, geometry_version)
```

The database does not need to carry the entire computational geometry payload inside the relational row.

---

19. MODEL VERSIONS

`revisions.model_versions`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_number INTEGER NOT NULL
parent_version_id UUID
created_by UUID NOT NULL
state_hash VARCHAR(128) NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(project_id, revision_number)
```

---

20. REVISIONS

`revisions.revisions`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_number INTEGER NOT NULL
parent_revision_id UUID
model_version_id UUID NOT NULL
reason TEXT
```

```
created_by UUID NOT NULL
status VARCHAR(32) NOT NULL
code_manifest_id UUID
validation_summary JSONB
created_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(project_id, revision_number)
```

---

21. CHANGE RECORDS

`revisions.change_records`

Fields:

```
id UUID PRIMARY KEY
revision_id UUID NOT NULL
object_id UUID NOT NULL
operation VARCHAR(64) NOT NULL
before_state JSONB
after_state JSONB
reason TEXT
source VARCHAR(64)
created_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_changes_revision
idx_changes_object
```

This table allows revision reconstruction and comparison.

---

22. ARIA COMMANDS

`model.aria_commands`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID
operation VARCHAR(64) NOT NULL
target_objects JSONB NOT NULL
parameters JSONB NOT NULL
preserve_constraints JSONB
reason TEXT
confidence NUMERIC
requested_by UUID NOT NULL
validation_requirements JSONB
```

```
status VARCHAR(32) NOT NULL
result JSONB
created_at TIMESTAMPTZ NOT NULL
completed_at TIMESTAMPTZ
```

Indexes:

```
idx_aria_commands_project
idx_aria_commands_revision
idx_aria_commands_status
```

---

23. DESIGN PROPOSALS

`model.design_proposals`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
description TEXT NOT NULL
affected_objects JSONB
proposed_changes JSONB NOT NULL
rationale TEXT
constraints JSONB
estimated_impacts JSONB
status VARCHAR(32) NOT NULL
created_by UUID NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

24. VIEWS

`documentation.views`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
view_type VARCHAR(64) NOT NULL
level_id UUID
section_plane JSONB
orientation JSONB
scale VARCHAR(64)
visibility_settings JSONB
model_version_id UUID NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

25. DRAWINGS

documentation.drawings

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
view_id UUID NOT NULL
drawing_number VARCHAR(64) NOT NULL
title VARCHAR(255) NOT NULL
discipline VARCHAR(64)
scale VARCHAR(64)
model_version_id UUID NOT NULL
revision_id UUID NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(project_id, drawing_number, revision_id)
```

---

26. SHEETS

documentation.sheets

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
sheet_number VARCHAR(64) NOT NULL
title VARCHAR(255) NOT NULL
discipline VARCHAR(64)
paper_size VARCHAR(32) NOT NULL
template_id UUID
revision VARCHAR(32)
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

27. SHEET-DRAWING ASSOCIATION

documentation.sheet_drawings

Fields:

```
sheet_id UUID NOT NULL
drawing_id UUID NOT NULL
```

```
position JSONB
viewport JSONB
display_settings JSONB
```

Primary key:

```
(sheet_id, drawing_id)
```

---

28. SHEET TEMPLATES

`documentation.sheet_templates`

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(255) NOT NULL
paper_size VARCHAR(32) NOT NULL
orientation VARCHAR(32)
title_block JSONB
graphic_standard JSONB
revision_block JSONB
version INTEGER NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

29. DIMENSIONS

`documentation.dimensions`

Fields:

```
id UUID PRIMARY KEY
view_id UUID NOT NULL
reference_objects JSONB NOT NULL
dimension_type VARCHAR(64) NOT NULL
value NUMERIC
unit VARCHAR(32)
tolerance JSONB
display_style JSONB
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

30. ANNOTATIONS

documentation.annotations

Fields:

```
id UUID PRIMARY KEY
view_id UUID NOT NULL
annotation_type VARCHAR(64) NOT NULL
text TEXT
target_object_id UUID
position JSONB
style JSONB
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

31. SCHEDULES

documentation.schedules

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
schedule_type VARCHAR(64) NOT NULL
source_objects JSONB NOT NULL
columns JSONB NOT NULL
sorting JSONB
filtering JSONB
view_id UUID
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

32. REDLINES

revisions.redlines

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
drawing_id UUID
target_object_id UUID
markup_geometry JSONB
instruction TEXT NOT NULL
priority VARCHAR(32)
created_by UUID NOT NULL
```

```
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

33. REDLINE RESOLUTIONS

`revisions.redline_resolutions`

Fields:

```
id UUID PRIMARY KEY
redline_id UUID NOT NULL
command_id UUID
result TEXT
implemented_changes JSONB
validation_results JSONB
resolved_by UUID
resolved_at TIMESTAMPTZ
```

---

34. CODE SOURCES

`compliance.code_sources`

Fields:

```
id UUID PRIMARY KEY
jurisdiction VARCHAR(255) NOT NULL
authority VARCHAR(255)
title VARCHAR(512) NOT NULL
source_uri TEXT
version VARCHAR(128)
effective_date DATE
retrieved_at TIMESTAMPTZ
content_hash VARCHAR(128)
status VARCHAR(32) NOT NULL
```

The source URI is informational and provenance-oriented; the retrieved source artifact should be stored through the document/object-storage subsystem where licensing permits.

---

35. CODE RULES

`compliance.code_rules`

Fields:

```
id UUID PRIMARY KEY
```

```
code_source_id UUID NOT NULL
rule_identifier VARCHAR(128) NOT NULL
citation VARCHAR(255)
rule_type VARCHAR(64)
requirement TEXT NOT NULL
applicability JSONB
threshold JSONB
unit VARCHAR(32)
exceptions JSONB
verification_method JSONB
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

36. CODE MANIFESTS

`compliance.code_manifests`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
jurisdiction VARCHAR(255) NOT NULL
municipality VARCHAR(255)
province_state VARCHAR(255)
country VARCHAR(128)
source_ids JSONB NOT NULL
source_versions JSONB NOT NULL
effective_dates JSONB
retrieval_timestamp TIMESTAMPTZ NOT NULL
content_hash VARCHAR(128) NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

A manifest becomes immutable once attached to a finalized revision.

---

37. VALIDATION RUNS

`validation.validation_runs`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
code_manifest_id UUID
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
status VARCHAR(32) NOT NULL
result_count INTEGER DEFAULT 0
```

```
summary JSONB
```

---

38. VALIDATION RESULTS

validation.validation_results

Fields:

```
id UUID PRIMARY KEY
validation_run_id UUID NOT NULL
project_id UUID NOT NULL
revision_id UUID NOT NULL
rule_id UUID
affected_objects JSONB
category VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
severity VARCHAR(32) NOT NULL
description TEXT NOT NULL
source JSONB
created_at TIMESTAMPTZ NOT NULL
```

Indexes:

```
idx_validation_project
idx_validation_revision
idx_validation_run
idx_validation_status
idx_validation_severity
```

---

39. DOCUMENT PACKAGES

documentation.document_packages

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
code_manifest_id UUID
file_references JSONB NOT NULL
document_hash VARCHAR(128)
generated_at TIMESTAMPTZ
status VARCHAR(32) NOT NULL
```

---

40. ACCEPTANCE

`revisions.acceptance_records`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
document_package_id UUID NOT NULL
code_manifest_id UUID
customer_id UUID NOT NULL
accepted_at TIMESTAMPTZ
acceptance_status VARCHAR(32) NOT NULL
acknowledgements JSONB
```

---

41. PROFESSIONAL REVIEW

`revisions.professional_reviews`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
reviewer_id UUID NOT NULL
professional_role VARCHAR(128)
review_scope TEXT
result VARCHAR(64) NOT NULL
comments TEXT
reviewed_at TIMESTAMPTZ
```

---

42. MUNICIPAL APPROVAL STATUS

`projects.approval_statuses`

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
authority VARCHAR(255) NOT NULL
application_reference VARCHAR(255)
status VARCHAR(64) NOT NULL
submitted_at TIMESTAMPTZ
decision_at TIMESTAMPTZ
notes TEXT
```

---

43. AUDIT EVENTS

```
audit.project_events
```

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
event_type VARCHAR(128) NOT NULL
actor_id UUID
revision_id UUID
object_id UUID
payload JSONB NOT NULL
timestamp TIMESTAMPTZ NOT NULL
```

The event table is append-only.

---

44. IDEMPOTENCY

All externally submitted mutating requests should support:

```
Idempotency-Key
```

The system stores:

```
request_hash
response_hash
response_payload
created_at
```

This prevents duplicate project mutations when clients retry requests.

---

45. OPENAPI ROOT

The machine API shall expose:

```
/api/v1
```

Metadata:

```
title: OneDraft API
version: 0.1.0
description: OneDraft CAD-AI project modeling and documentation API
```

---

46. AUTHENTICATION

The MVP should use bearer-token authentication.

Conceptually:

```
Authorization: Bearer <token>
```

The API gateway validates the token before the request reaches project services.

---

47. AUTHORIZATION SCOPES

Initial scopes:

```
project:read
project:write

model:read
model:write

aria:execute
aria:approve

drawing:read
drawing:generate

redline:read
redline:write

compliance:read
compliance:execute

validation:read
validation:execute

document:read
document:generate

acceptance:write

admin:organization
admin:project
```

The API must evaluate both:

scope

and:

organization/project membership

---

48. STANDARD RESPONSE

Successful responses use:

```
{
  "data": {},
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

Collection responses additionally contain:

```
{
  "data": [],
  "metadata": {
    "request_id": "uuid",
    "page": 1,
    "page_size": 50,
    "total": 100
  }
}
```

---

49. STANDARD ERROR

```
{
  "error": {
    "code": "CONSTRAINT_CONFLICT",
    "message": "Requested operation conflicts with an active project constraint.",
    "object_id": "uuid",
    "resolution": "Review the affected constraint or submit a revised command."
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

---

50. PROJECT CREATION REQUEST

Conceptually:

```
{
  "name": "Clark Residence",
  "project_type": "RESIDENTIAL",
  "description": "Two-storey residence",
  "address": {
    "street": "...",
    "city": "...",
```

```
    "province": "...",
    "country": "...",
    "postal_code": "..."
  },
  "unit_system": "METRIC"
}
```

The API returns:

```
Project_ID
initial Revision_ID
initial Model_Version_ID
```

---

51. ARIA REQUEST

Endpoint:

```
POST /api/v1/projects/{project_id}/aria/requests
```

Request:

```
{
  "message": "Move the rear bedroom wall 300 mm north while keeping the exterior
envelope unchanged.",
  "revision_id": "uuid",
  "mode": "DESIGN_MODIFICATION"
}
```

Response:

```
{
  "data": {
    "command_id": "uuid",
    "status": "PROPOSED",
    "operation": "MOVE_ELEMENT",
    "targets": ["uuid"],
    "parameters": {
      "distance": 300,
      "unit": "mm",
      "direction": "NORTH"
    }
  }
}
```

---

52. COMMAND REQUEST

```
{
  "operation": "MOVE_ELEMENT",
  "target_objects": [
    "uuid"
```

```
  ],
  "parameters": {
    "distance": 300,
    "unit": "mm",
    "direction": "NORTH"
  },
  "preserve_constraints": [
    "uuid"
  ],
  "expected_revision": 14
}
```

The server verifies that Revision 14 remains current.

---

53. COMMAND RESPONSE

```
{
  "data": {
    "command_id": "uuid",
    "status": "COMPLETED",
    "revision_id": "uuid",
    "model_version_id": "uuid",
    "affected_objects": [
      "uuid",
      "uuid"
    ],
    "affected_drawings": [
      "A101",
      "A201",
      "A401"
    ]
  }
}
```

---

54. REDLINE REQUEST

```
{
  "drawing_id": "uuid",
  "target_object_id": "uuid",
  "instruction": "Add a 900 mm wide window centered on this wall.",
  "markup_geometry": {},
  "priority": "NORMAL"
}
```

The redline initially receives:

```
OPEN
```

It becomes:

`IMPLEMENTED`

only after a model operation has been successfully executed.

---

55. VALIDATION REQUEST

`POST /api/v1/projects/{project_id}/validation`

Request:

```
{
  "revision_id": "uuid",
  "code_manifest_id": "uuid",
  "categories": [
    "CODE",
    "SPATIAL",
    "ACCESSIBILITY",
    "DOCUMENT",
    "COORDINATION"
  ]
}
```

Response:

```
{
  "data": {
    "validation_run_id": "uuid",
    "status": "QUEUED"
  }
}
```

Validation executes asynchronously.

---

56. DRAWING GENERATION REQUEST

`POST /api/v1/projects/{project_id}/drawings/generate`

Request:

```
{
  "revision_id": "uuid",
  "drawing_types": [
    "PLAN",
    "ELEVATION",
    "SECTION"
  ],
  "sheet_template": "ARCH_D"
}
```

Response:

```
{
  "data": {
    "job_id": "uuid",
    "status": "QUEUED"
  }
}
```

---

57. DOCUMENT PACKAGE REQUEST

`POST /api/v1/projects/{project_id}/documents/generate`

Request:

```
{
  "revision_id": "uuid",
  "format": "PDF",
  "paper_size": "ARCH_D",
  "include": [
    "PLANS",
    "ELEVATIONS",
    "SECTIONS",
    "SCHEDULES",
    "NOTES"
  ]
}
```

---

58. API ASYNCHRONOUS JOB MODEL

Expensive operations shall return a Job_ID.

Initial job statuses:

```
QUEUED
RUNNING
VALIDATING
COMPLETED
FAILED
CANCELLED
```

Job endpoint:

`GET /api/v1/jobs/{job_id}`

Response:

```
{
  "data": {
    "job_id": "uuid",
    "type": "DOCUMENT_GENERATION",
    "status": "RUNNING",
```

```
    "progress": 72
  }
}
```

---

59. PAGINATION

Collection endpoints shall use:

```
page
page_size
```

Future versions may add cursor pagination.

The initial maximum:

```
page_size <= 100
```

---

60. FILTERING

Collections should support:

```
status
type
created_after
created_before
updated_after
updated_before
```

Model elements additionally support:

```
level_id
element_type
parent_id
```

---

61. API VERSIONING

The initial API is:

```
/api/v1
```

Breaking changes require:

```
/api/v2
```

Non-breaking additions remain within the current version.

---

62. DATABASE TRANSACTION BOUNDARY

A command modifying the model must execute within an atomic database transaction.

Conceptually:

```
BEGIN

verify project
verify revision
verify authorization
verify constraints
apply model mutation
record change
create revision
record event

COMMIT
```

If any required operation fails:

```
ROLLBACK
```

No partial model state may remain.

---

63. GEOMETRY TRANSACTION BOUNDARY

Because geometry computation may be expensive, the architecture separates:

database transaction

from:

geometry worker execution

The workflow becomes:

```
COMMAND
  ↓
MODEL TRANSACTION
  ↓
GEOMETRY JOB
  ↓
GEOMETRY VALIDATION
  ↓
MODEL FINALIZATION
```

The system must ensure the resulting geometry corresponds to the exact model version for which it was generated.

---

64. GEOMETRY VERSION CHECK

Every geometry job records:

```
project_id
model_version_id
element_id
geometry_version
```

If the model changes while a geometry job is running, the job may be invalidated.

This prevents stale geometry from becoming current project state.

---

65. DRAWING VERSION CHECK

Likewise, every drawing job records:

```
project_id
model_version_id
revision_id
view_id
```

A drawing generated from Revision 17 must never be silently presented as the drawing for Revision 18.

---

66. DOCUMENT VERSION CHECK

Every final document package must identify:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
```

This creates a reproducible chain:

```
PROJECT
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
CODE MANIFEST
```

```
 ↓
VALIDATION
 ↓
DRAWING SET
 ↓
DOCUMENT PACKAGE
```

---

67. DATABASE INDEX STRATEGY

High-frequency access patterns must receive explicit indexes.

Primary targets:

```
project_id
organization_id
level_id
building_id
element_type
parent_id
revision_id
model_version_id
status
created_at
updated_at
```

The database should be profiled before adding large numbers of speculative indexes.

---

68. JSONB STRATEGY

JSONB is permitted for:

AI parameters

geometry metadata

complex constraints

configuration

regulatory applicability

document metadata

rendering settings

JSONB shall not replace relational columns for fields requiring:

frequent joins

foreign-key integrity

high-frequency filtering

unique constraints

The semantic model remains relational where relationships matter.

---

69. DATABASE MIGRATION ORDER

The initial migration sequence should be:

```
001_extensions
002_identity
003_projects
004_requirements
005_buildings
006_levels
007_sites
008_spaces
009_assemblies
010_elements
011_constraints
012_relationships
013_geometry
014_model_versions
015_revisions
016_commands
017_documentation
018_redlines
019_compliance
020_validation
021_documents
022_acceptance
023_events
024_indexes
025_seed_data
```

Each migration is immutable once applied.

---

70. SEED DATA

Development environments should contain:

ARCH D sheet template

basic wall assembly

basic door

basic window

residential project type

commercial project type

initial element types

initial command types

test jurisdiction

The test jurisdiction must be clearly marked as development data.

---

71. OPENAPI RESOURCE GROUPS

The API specification shall be organized under:

```
Projects
Organizations
Requirements
Buildings
Levels
Spaces
Elements
Constraints
Commands
Revisions
Redlines
Views
Drawings
Sheets
Documents
Compliance
Validation
Exports
Jobs
Acceptance
Events
```

---

72. MACHINE-READABLE CONTRACT

The final implementation repository shall contain:

```
/docs/openapi/openapi.yaml
```

This file becomes the machine-readable contract between:

frontend

backend

workers

external integrations

future mobile clients

and potentially:

third-party professional applications

---

73. GENERATED CLIENTS

Once the OpenAPI contract stabilizes, typed clients can be generated for:

TypeScript

Python

and other supported languages.

The frontend should consume the generated TypeScript API client rather than manually duplicating endpoint definitions.

---

74. DATABASE / API SEPARATION

External clients must never connect directly to PostgreSQL.

The architecture remains:

```
CLIENT
 ↓
API
 ↓
DOMAIN SERVICES
 ↓
DATABASE
```

Workers may access internal services through controlled interfaces.

---

75. AI PROVIDER SEPARATION

The API contract does not expose the underlying AI provider.

Externally:

```
OneDraft → Aria
```

Internally:

```
Aria Orchestration
 ↓
```

```
AI Provider Adapter
 ↓
Model Provider
```

This allows the underlying model infrastructure to evolve without redesigning the OneDraft API.

---

76. AI COMMAND SAFETY BOUNDARY

The following are prohibited as direct AI actions:

```
direct SQL
direct database writes
direct file replacement
direct production configuration modification
direct permission changes
direct billing modification
```

Aria must operate through defined application capabilities.

---

77. SYSTEM OF RECORD HIERARCHY

The OneDraft system of record is ordered:

1. Project Model

2. Revision History

3. Regulatory Manifest

4. Validation Results

5. Generated Documentation

6. Customer Acceptance

7. Audit Events

Derived documentation can always be regenerated from the appropriate model state.

---

78. RECONSTRUCTION TEST

OneDraft must eventually pass the following test:

Given:

```
Project_ID
Revision_ID
```

```
Model_Version_ID
Code_Manifest_ID
```

the system must be capable of reconstructing the project state and regenerating its associated documentation.

If this cannot be done, the provenance architecture is incomplete.

---

79. MVP DATABASE SUCCESS CRITERIA

The database implementation is successful when it can:

Create a project

Store requirements

Create a building

Create levels

Create spaces

Create architectural elements

Store relationships

Store constraints

Version model state

Record Aria commands

Record revisions

Generate drawing references

Record redlines

Attach code manifests

Store validation results

Create document packages

Record customer acceptance

Reconstruct historical state

---

80. MVP API SUCCESS CRITERIA

The API implementation is successful when a client can execute:

```
CREATE PROJECT
 ↓
CREATE SPECIFICATION
 ↓
CREATE MODEL
 ↓
MODIFY MODEL THROUGH ARIA
 ↓
GENERATE DRAWINGS
 ↓
RUN VALIDATION
 ↓
CREATE REDLINE
 ↓
RESOLVE REDLINE
 ↓
REGENERATE DRAWINGS
 ↓
GENERATE FINAL DOCUMENT PACKAGE
 ↓
ACCEPT
```

without directly accessing the database.

---

81. FIRST AUTOMATED INTEGRATION TEST

The first complete integration test should execute:

```
1. Create organization

2. Create user

3. Create project

4. Create requirements

5. Create building

6. Create Ground Level

7. Create Level 1

8. Create rooms

9. Create walls

10. Create doors
```

11. Create windows
12. Generate plans
13. Generate elevations
14. Generate section
15. Generate ARCH D sheets
16. Submit Aria modification
17. Validate command
18. Create Revision 2
19. Regenerate affected drawings
20. Create redline
21. Resolve redline
22. Run validation
23. Generate document package
24. Hash document package
25. Record customer acceptance
26. Retrieve complete project history

This test becomes the first major OneDraft regression test.

---

82. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now has:

Product definition

System architecture

Technology architecture

Domain model

API contract

Persistence model

Versioning model

AI command boundary

Regulatory model

Drawing model

Redline model

Validation model

Document provenance

Acceptance workflow

The remaining step before implementation is to convert this specification into executable artifacts.

---

83. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package should consist of four machine-oriented artifacts:

Artifact A

`001_initial_schema.sql`

The first PostgreSQL migration.

Artifact B

`openapi.yaml`

The initial OpenAPI 3 specification.

Artifact C

`domain_types.py`

Python/Pydantic domain and API models.

Artifact D

`project_model_service.py`

The first implementation of the Project Model command boundary.

Together these four artifacts create the first executable OneDraft foundation.

---

84. FINAL ARCHITECTURAL DECISION

OneDraft shall not be developed as:

```
CHATBOT
+
IMAGE GENERATOR
```

```
+
PDF EXPORT
```

It shall be developed as:

```
COMPUTATIONAL PROJECT MODEL
+
ARIA INTELLIGENCE
+
GEOMETRY ENGINE
+
REGULATORY ENGINE
+
DOCUMENT ENGINE
+
VERSION CONTROL
+
CUSTOMER REVIEW
```

The architectural drawing is therefore a **derived product of the computational project model**, rather than the model itself.

That distinction is fundamental to the OneDraft platform.

---

85. SPECIFICATION STATUS

ONEDRAFT DATABASE SCHEMA & OPENAPI CONTRACT v0.1

STATUS: ESTABLISHED

The next implementation stage is the creation of the four executable foundation artifacts:

`001_initial_schema.sql`

`openapi.yaml`

`domain_types.py`

`project_model_service.py`

These artifacts will constitute the first actual OneDraft software skeleton.